Dokumentacja Techniczna i Architektoniczna Systemu Audytowego `sys-auditd`

Pełna Specyfikacja Architektury Oprogramowania, Modułów C, Protokołu i Mechanizmów Bezpieczeństwa

Zespół Bezpieczeństwa Systemów Linux i Inżynierii Oprogramowania

17 października 2026

Spis treści

1 Wstęp i Założenia Projektowe **2**
- 1.1 Cel i Zakres Systemu . . . 2
- 1.2 Kluczowe Paradygmaty Architektoniczne . . . 2

2 Architektura Systemowa i Przepływ Danych **3**
- 2.1 Cykl Życia Zdarzenia Audytowego . . . 3

3 Szczegółowa Architektura Modułów C **4**
- 3.1 Moduł `config_5.c` – Odkrywanie Zasobów i Parsowanie . . . 4
- 3.2 Moduł `connections_3.c` – Inspekcja Połączeń TCP . . . 4
- 3.3 Moduł `diff_2.c` – Silnik Porównywania Tekstu . . . 4
- 3.4 Moduł `heartbeat_4.c` – Sygnał Życia . . . 4
- 3.5 Moduł `io_2.c` – Bezpieczne Operacje We/Wy . . . 5
- 3.6 Moduł `main_5.c` – Pętla Główna i Maskowanie . . . 5
- 3.7 Moduł `monitor_5.c` – Obsługa Inotify i Logów . . . 5
- 3.8 Moduł `network_3.c` – Analiza Kontekstu Sieciowego . . . 5
- 3.9 Moduł `port_5.c` – Wysyłka Raportów HTTPS . . . 5
- 3.10 Moduł `protocol_5.c` – Serializacja i JSON . . . 6
- 3.11 Moduł `snapshot_5.c` – Zarządzanie Migawkami . . . 6

4 Format Raportów Zdarzeń JSON **7**
- 4.1 Przykładowy Raport Modyfikacji Pliku (Unified Diff) . . . 7
- 4.2 Przykładowy Raport Połączenia SIECIOWEGO TCP . . . 7

5 Mechanizmy Bezpieczeństwa i Higiena Pamięci **9**
- 5.1 Czyszczenie Pamięci Operacyjnej (Memory Hygiene) . . . 9
- 5.2 Atomowość Operacji Plikowych . . . 9
- 5.3 Restrykcje Uprawnień . . . 9

1 Wstęp i Założenia Projektowe

1.1 Cel i Zakres Systemu

System `sys-auditd` jest natywnym demonem audytowym dla systemów operacyjnych Linux, zaprojektowanym do ciągłego monitorowania integralności plików (File Integrity Monitoring – FIM), analizy logów zdarzeń oraz inspekcji gniazd sieciowych w czasie rzeczywistym. System kładzie szczególny nacisk na audyt środowisk udostępniania plików opartych na serwerze Samba (udziały sieciowe, bazy kont `tdbsam` oraz `smbpasswd`) oraz powiązanych logów systemowych i sieciowych.

Głównym celem projektowym jest zapewnienie maksymalnej niezawodności, odporności na manipulację ze strony napastnika posiadającego uprawnienia w warstwie użytkownika oraz ultrawydajności I/O bez obciążania zasobów procesora.

1.2 Kluczowe Paradygmaty Architektoniczne

- **Model Bezprocesowy (No-Fork / No-Exec):** System nie tworzy procesów potomnych poprzez `fork()` ani nie wykonuje zewnętrznych binarów (`exec()`). Wszystkie funkcje – w tym parsowanie konfiguracji Samba, inspekcja baz TDB, wyznaczanie diffów oraz obsługa gniazd – są realizowane natywnie wewnątrz jednego procesu. Eliminacja wywołań `fork/exec` zapobiega generowaniu śladów w podsystemach audytowych jądra (`auditd`, eBPF) oraz eliminuje narzut przełączania kontekstu.

- **Izolowane We/Wy (Isolated I/O z `openat2`):** Operacje na strukturze katalogów i plików opierają się na wywołaniu systemowym `SYS_openat2` z flagami `RESOLVE_BENEATH`, `RESOLVE_NO_SYMLINKS` oraz `RESOLVE_NO_MAGICLINKS`. Gwarantuje to, że żaden odczyt ani zapis nie opuści wskazanego poddrzewa katalogów, wykluczając podatności typu TOCTOU (Time-of-Check to Time-of-Use) oraz ataki z wykorzystaniem dowiązań symbolicznych i sztywnych.

- **Wielowątkowość z podziałem odpowiedzialności:** Pętla główna demona realizuje współbieżną obsługę zdarzeń `inotify`, inspekcję gniazd Netlink `SOCK_DIAG` oraz skanowanie przyrostowe logów. Osobny wątek nadawczy (`sender_loop`) odpowiada za asynchroniczną i nieblokującą wysyłkę buforowanych raportów przez połączenie HTTPS/TLS.

- **Korelacja kontekstu w czasie rzeczywistym:** Zdarzenia modyfikacji plików są automatycznie wiązane z logami aplikacyjnymi zarejestrowanymi w ciągu ostatnich 60 sekund oraz z danymi adresowymi pobieranymi bezpośrednio z lokalnej tablicy ARP jądra (adres IP, adres MAC, interfejs sieciowy).

2 Architektura Systemowa i Przepływ Danych

2.1 Cykl Życia Zdarzenia Audytowego

Przepływ informacji od momentu detekcji incydentu do jego trwałego zatwierdzenia na serwerze centralnym przebiega według następujących kroków:

1. **Detekcja:** Podsystem `inotify` zgłasza zmianę w pliku lub podsystem Netlink `SOCK_DIAG` wykrywa nowe połączenie TCP w stanie `ESTABLISHED`.
2. **Weryfikacja i Capture:** Moduł `snapshot_5.c` wykonuje 3-etapową weryfikację spójności pliku (`capture()`), sprawdzając tożsamość metadanych przed i po odczycie, po czym wykonuje kopię w bezpiecznym katalogu roboczym.
3. **Analiza Różnicowa (Diffing):** Moduł `diff_2.c` porównuje nowy stan pliku z poprzednio zapamiętaną migawką. Dla plików tekstowych UTF-8 generowany jest unifikowany diff (Unified Diff) przy użyciu algorytmu LCS Myersa. Dla plików binarnych stosowany jest diff blokowy 64 KB.
4. **Dołączanie Kontekstu i Dowodów:** Moduł `network_3.c` dołącza wpisy z tablicy ARP jądra dla adresów IP wykrytych w treści zdarzenia, a bufor zdarzeń logów dołącza pasujące wpisy z ostatnich 60 sekund.
5. **Atomowy Zapis Raportu:** Moduł `protocol_5.c` formatuje raport w formacie JSON, który jest zapisywany w katalogu `pending/` z użyciem wywołania `renameat2(..., RENAME_NOREPLACE)`.
6. **Asynchroniczna Transmisja:** Wątek nadawczy wysyła buforowany raport do serwera zbiorczego przez HTTPS (`libcurl`). Po odebraniu potwierdzenia HTTP 2xx raport jest trwale usuwany.

3 Szczegółowa Architektura Modułów C

Podsystem składa się z 11 zintegrowanych modułów C. Poniższa sekcja przedstawia szczegółowy opis odpowiedzialności i implementacji każdego z nich.

3.1 Moduł `config_5.c` – Odkrywanie Zasobów i Parsowanie

Moduł realizuje automatyczną detekcję konfiguracji Samba, baz kont oraz logów systemowych:

- **Parsowanie `smb.conf`:** Bezpośrednia interpretacja składni plików INI Samby, sekcji `[global]`, sekcji udziałów, dyrektyw `include` oraz parametrów `path`, `available`, `printable`.
- **Natywna obsługa bazy `tdbsam`:** Funkcja `read_tdb()` otwiera plik `passdb.tdb` za pomocą interfejsu `libtdb` (`tdb_open`) na wyizolowanym deskryptorze z blokadą `F_RDLCK`. Iteracja po kluczach `USER_*` pozwala wypakować listę użytkowników bez uruchamiania zewnętrznych procesów `pdbedit`.
- **Natywna obsługa `smbpasswd`:** Funkcja `read_smbpasswd()` umożliwia parsowanie tradycyjnych baz tekstowych haseł Samby.
- **Dynamiczne udziały domowe (`homes`):** Funkcja `resolve_homes()` mapuje udziały typu `homes` na ścieżki katalogów domowych użytkowników pobierane z bazy systemowej (`getpwnam_r`).

3.2 Moduł `connections_3.c` – Inspekcja Połączeń TCP

Moduł odpowiada za bezprocesowy odczyt stanu sieci z jądra Linux:

- **Netlink `SOCK_DIAG`:** Funkcja `collect_family()` komunikuje się z gniazdem `NETLINK_SOCK_DIAG`, pobierając tablicę gniazd TCP dla rodzin `AF_INET` i `AF_INET6`.
- **Weryfikacja procesów nasłuchujących:** Funkcja `has_listener()` sprawdza, czy obserwowane gniazdo w stanie `TCP_ESTABLISHED` odpowiada portowi, na którym lokalnie nasłuchuje usługa w stanie `TCP_LISTEN`.
- **Limit pamięci:** Liczba jednocześnie śledzonych gniazd ograniczona jest stałą `HP_SOCKET_LIMIT` (65 536 gniazd).

3.3 Moduł `diff_2.c` – Silnik Porównywania Tekstu

Moduł wyznacza różnice w plikach tekstowych bezpośrednio w pamięci RAM:

- **Algorytm Myers LCS:** Funkcja `hp_write_text_diff()` wyznacza Najdłuższy Wspólny Podciąg (LCS) dla linii tekstu zakodowanego w UTF-8. Rozmiar analizowanych plików ograniczony jest do 1 MB (`HP_TEXT_LIMIT`), a macierz LCS do 10^6 komórek (`HP_LCS_CELLS`).
- **Przełączenie na diff blokowy:** W przypadku plików binarnych (obecność bajtu `0x00` lub błędnego UTF-8) lub zbyt dużej liczby zmian, system przełącza się na blokowe porównanie Base64 (64 KB) zrealizowane w `snapshot_5.c`.

3.4 Moduł `heartbeat_4.c` – Sygnał Życia

- **Losowe odchylenie czasowe (Jitter):** Funkcja `hp_heartbeat_tick()` wyznacza czas kolejnego sygnału kontrolnego z użyciem kryptograficznego generatora jądra `getrandom()`. Sygnał generowany jest w interwale $[0.85, 1.15] \times$ HEARTBEAT_INTERVAL, co uniemożliwia wykrycie działania demona poprzez analizę wzorców czasowych (anti-beaconing).

3.5 Moduł `io_2.c` – Bezpieczne Operacje We/Wy

Zapewnia warstwę abstrakcji I/O odporną na ataki w strukturze plików:

- **`hp_open_beneath()`:** Używa natywnego wywołania `SYS_openat2` z flagami `RESOLVE_BENEATH | RESOLVE_NO_SYMLINKS | RESOLVE_NO_MAGICLINKS`.
- **`hp_private_file()` i `hp_private_directory()`:** Tworzy i weryfikuje katalogi/pliki robocze, sprawdzając właściciela (`st_uid == geteuid()`), uprawnienia (`st_mode & 077 == 0`) oraz brak dodatkowych dowiązań (`st_nlink == 1`).
- **`hp_read_file()`:** Wykonuje całościowy odczyt pliku z weryfikacją znaczników czasu i rozmiaru `fstat` przed i po odczycie.

3.6 Moduł `main_5.c` – Pętla Główna i Maskowanie

- **Maskowanie procesu:** Ustawienie nazwy wątku przez `prctl(PR_SET_NAME, "sys-auditd")` oraz nadpisanie bufora `argv[0]` ciągiem kamuflującym.
- **Obsługa sygnałów:** Bezpieczna obsługa sygnałów `SIGINT` i `SIGTERM` za pomocą zmiennej atomowej `atomic_int stopping` bez użycia blokad.
- **Dynamiczna rekonfiguracja:** Co 60 sekund pętla główna weryfikuje strukturę źródeł logów i udziałów. W przypadku zmian następuje przeładowanie struktur bez przerywania pracy demona.

3.7 Moduł `monitor_5.c` – Obsługa Inotify i Logów

- **Subskrypcja zdarzeń `inotify`:** Skanowanie drzewa katalogów i montowanie obserwacji na zdarzenia `IN_CLOSE_WRITE`, `IN_CREATE`, `IN_DELETE`, `IN_MOVED_FROM`, `IN_MOVED_TO`, `IN_ATTRIB`.
- **Śledzenie rotacji logów:** Polling logów (`poll_logs`) wykrywa rotację plików (zmiana `inode` lub `st_dev`) oraz ich skrócenie (`st_size < offset`), dbając o ciągłość odczytu i obsługę niepełnych rekordów.
- **Skojarzenie operacji przeniesienia:** Kolejki `moves` i `created` parują zdarzenia `IN_MOVED_FROM` z `IN_MOVED_TO` na podstawie unikalnego ciasteczka (`cookie`).

3.8 Moduł `network_3.c` – Analiza Kontekstu Sieciowego

- **`hp_extract_ips()`:** Ekstrakcja adresów IPv4 oraz IPv6 z ciągów tekstowych zdarzeń.
- **`hp_lookup_mac()`:** Wykonanie zapytania `ioctl(SIOCGARP)` dla wykrytych adresów IP w celu pobrania adresu MAC oraz nazwy interfejsu z lokalnej tablicy ARP jądra.

3.9 Moduł `port_5.c` – Wysyłka Raportów HTTPS

- **Transmisja przez `libcurl`:** Asynchroniczna wysyłka raportów z katalogu `pending/` metodą HTTP POST pod adres `https://<IP>:<PORT>/events`.
- **Rygorystyczny TLS:** Wymuszone opcje `CURLOPT_SSL_VERIFYPEER = 1L` oraz `CURLOPT_SSL_VERIFYHOST = 2L`.
- **Zatwierdzanie:** Po odebraniu odpowiedzi HTTP 2xx raport jest usuwany z dysku (`unlinkat()`), po czym wykonywane jest `hp_sync()`.

3.10 Moduł `protocol_5.c` – Serializacja i JSON

- **Walidacja UTF-8:** Weryfikacja poprawności sekwencji bajtów UTF-8 (`hp_valid_utf8()`).
- **Kodowanie Base64:** Użycie funkcji `EVP_EncodeBlock()` z biblioteki OpenSSL dla danych binarnych.
- **Identyfikatory i czas:** Generowanie identyfikatorów za pomocą `RAND_bytes()` oraz znaczników czasu ISO-8601 UTC przez `gmtime_r()`.

3.11 Moduł `snapshot_5.c` – Zarządzanie Migawkami

- **Potrójne pobieranie (`capture()`):** Do 3 prób skopiowania pliku z weryfikacją spójności metadanych `fstat` (mtime, ctime, inode) przed i po odczycie.
- **Suma kontrolna SHA-256:** Obliczanie skrótów SHA-256 przy użyciu OpenSSL EVP API.
- **Rejestr metadanych:** Zapis metadanych (`uid`, `gid`, `mode`, `mtime`) w plikach `.stat` umożliwia detekcję zmian samych uprawnień pliku.
- **Rekoncyliacja przestoju:** Po uruchomieniu demon porównuje stan migawek ze stanem na dysku i generuje raporty dla plików usuniętych lub zmienionych podczas nieaktywności demona.

4 Format Raportów Zdarzeń JSON

4.1 Przykładowy Raport Modyfikacji Pliku (Unified Diff)


```
{
  "schema_version": 1,
  "report_id": "a8f3d1b2e4c567890123456789abcdef0123456789abcdef0123456789abcdef",
  "agent_id": "srv-samba-01",
  "observed_at": "2026-10-17T17:04:00Z",
  "source": "share",
  "operation": "modify",
  "path": "/srv/samba/finance/budget.txt",
  "metadata": {
    "old_size": 128,
    "new_size": 142,
    "old_mode": 420,
    "new_mode": 420,
    "old_uid": 1001,
    "new_uid": 1001,
    "old_gid": 1000,
    "new_gid": 1000,
    "mtime_sec": 1792256640
  },
  "analysis": {
    "classification": "file_change",
    "actor": null
  },
  "evidence": [
    {
      "source": "/var/log/samba/log.smbd",
      "context_only": true,
      "content": "smbd_audit: user [john] modified [finance/budget.txt]",
      "network_context": {
        "endpoints": [
          {
            "ip": "192.168.1.50",
            "mac": "00:11:22:33:44:55",
            "interface": "eth0",
            "mac_status": "resolved",
            "mac_source": "local_arp_cache_at_report_time"
          }
        ],
        "truncated": false
      }
    }
  ],
  "content_changes": {
    "old_sha256": "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855",
    "new_sha256": "f2ca1bb6c7e907d06dafe4687e579fce76b37e4e93b7605022da52e6ccc26fd2",
    "format": "unified",
    "diff": "--- old\n+++ new\n@@ -1,2 +1,2 @@\n Status: Draft\n-Total: 5000\n+Total:
    7500\n"
  },
  "complete": true,
  "error": null
}
```


4.2 Przykładowy Raport Połączenia SIECIOWEGO TCP

```
{
  "schema_version": 1,
  "report_id": "b9e4f2a1c3d567890123456789abcdef0123456789abcdef0123456789abcdef",
  "agent_id": "srv-samba-01",
  "observed_at": "2026-10-17T17:05:12Z",
  "source": "network",
  "operation": "tcp_connection_observed",
  "path": "",
  "metadata": {},
  "analysis": {
    "classification": "tcp_connection",
    "actor": null
  },
  "evidence": [],
  "content_changes": null,
  "complete": true,
  "error": null,
  "connection": {
    "protocol": "tcp",
    "state": "ESTABLISHED",
    "direction": "inbound_listener_match",
    "observation_method": "inet_diag_poll",
    "local_ip": "192.168.1.10",
    "local_port": 445,
    "peer_port": 52134,
    "peer": {
      "ip": "192.168.1.50",
      "mac": "00:11:22:33:44:55",
      "interface": "eth0",
      "mac_status": "resolved",
      "mac_source": "local_arp_cache_at_report_time",
      "context_only": true
    }
  }
}
```

5 Mechanizmy Bezpieczeństwa i Higiena Pamięci

5.1 Czyszczenie Pamięci Operacyjnej (Memory Hygiene)

System konsekwentnie czyści bufory pamięci zawierające dane wrażliwe za pomocą funkcji `explicit_bzero()`, co zapobiega ich wyciekowi do plików wymiany (swap) lub w przypadku wykonania zrzutu pamięci (core dump):

- Wymazywanie odczytanych fragmentów logów oraz migawek plików.
- Wymazywanie struktur haseł i kont pobieranych z `passdb.tdb` oraz `smbpasswd`.
- Wymazywanie kluczy i buforów pomocniczych kontekstów OpenSSL EVP.
- Wymazywanie buforów odczytu gniazd Netlink.

5.2 Atomowość Operacji Plikowych

Użycie wywołania `renameat2()` z flagą `RENAME_NOREPLACE` gwarantuje, że pliki tymczasowe raportów są przenoszone do katalogu wysyłkowego w sposób atomowy. W przypadku istnienia pliku docelowego operacja zwraca błąd bez naruszania istniejącej struktury danych.

5.3 Restrykcje Uprawnień

Przed uruchomieniem pętli zdarzeń demon wymusza maskę praw tworzenia plików `umask(0077)`. Katalogi robocze `snapshots/` oraz `pending/` są tworzone z uprawnieniami `0700`, a pliki raportów z uprawnieniami `0600`.